TP MI-GUIDE INSTALLATION ET ARCHITECTURE DE BASE DE DJANGO

Nom : Aya Lmasoudi

Lien du dépôt GitHub : https://github.com/lmasoudiaya-11/Exs_python/tree/main/Tp_3

Introduction :

Dans ce travail pratique, nous allons découvrir les bases nécessaires pour commencer à développer une application web en utilisant le framework Django. Django est un framework web open-source développé en Python qui permet de créer des applications web de manière plus rapide et structurée. Il est reconnu pour sa robustesse, sa sécurité et sa capacité à faciliter le développement d'applications web complexes.

Plusieurs plateformes connues ont utilisé Django dans leurs projets, notamment Instagram dans ses premières versions, Pinterest ou encore Mozilla. Cela montre que ce framework est capable de supporter des applications utilisées par un très grand nombre d'utilisateurs.

L'un des principaux avantages de Django est qu'il propose une architecture prête à l'emploi qui simplifie le travail des développeurs. Il fournit différents outils permettant de gérer facilement plusieurs aspects d'une application web, comme la gestion des URL, la communication avec les bases de données, la création de pages web à l'aide de templates HTML, l'authentification des utilisateurs, la sécurité ainsi que l'administration du site grâce à une interface intégrée.

Avant de commencer à développer une application avec Django, il est nécessaire de préparer un environnement de développement adapté. Ce TP nous permet donc de nous familiariser avec les étapes essentielles de cette préparation ainsi qu'avec la structure d'un projet Django.

Objectifs du TP :

L'objectif principal de ce TP est de comprendre les étapes nécessaires pour mettre en place un environnement de développement Django et créer une première application web. À travers ce travail pratique, nous allons apprendre à installer les outils nécessaires et à utiliser les commandes permettant de créer et d'exécuter un projet Django.

Plus précisément, ce TP vise à :

- Installer Django à l'aide de l'outil pip
- Comprendre l'utilité et le rôle des environnements virtuels
- Créer un projet Django
- Comprendre la structure d'un projet Django
- Lancer un serveur de développement local
- Identifier le rôle des fichiers générés automatiquement par Django

Installation de Python :

Avant de commencer à travailler avec Django, il est nécessaire de vérifier si Python est déjà installé sur la machine. Python est le langage de programmation utilisé par Django, donc son installation est indispensable pour pouvoir créer et exécuter un projet Django.

La première étape consiste à ouvrir un terminal (ou une invite de commandes) et à exécuter une commande permettant d'afficher la version de Python installée sur l'ordinateur. Pour cela, on utilise la commande suivante :

python --version

Dans certains systèmes, la commande suivante peut également être utilisée :

python3 --version

Si Python est correctement installé sur la machine, le terminal affiche la version actuellement installée. Par exemple, un message similaire au suivant peut apparaître : Python 3.11.4

```
TERMINAL
PS C:\Users\DELL\Desktop\Tp_3> python --version
Python 3.14.3
```

Ce résultat confirme que Python est bien installé et prêt à être utilisé pour la suite du TP. En revanche, si la commande n'est pas reconnue par le système, cela signifie que Python n'est pas encore installé sur l'ordinateur et qu'il faut alors procéder à son installation avant de continuer.

Création d'un environnement virtuel :

Dans un projet Python, il est fréquent d'utiliser plusieurs bibliothèques. Cependant, différents projets peuvent nécessiter des versions différentes d'une même bibliothèque. Sans environnement virtuel, ces versions peuvent entrer en conflit.

L'utilisation d'un environnement virtuel permet d'isoler les dépendances de chaque projet, d'éviter les conflits entre bibliothèques et de reproduire facilement le même environnement de développement.

Pour créer un environnement virtuel, on ouvre d'abord un terminal puis on se place dans le dossier du projet. Ensuite, on exécute la commande suivante :

```
python -m venv monenv
```

Cette commande crée un environnement virtuel nommé monenv. Après sa création, il faut l'activer avec la commande suivante :

```
monenv\Scripts\activate
```

```
PS C:\Users\DELL\Desktop\Tp_3> python -m venv env
PS C:\Users\DELL\Desktop\Tp_3> env\Scripts\activate
(env) PS C:\Users\DELL\Desktop\Tp_3>
```

Une fois l'environnement activé, toutes les bibliothèques installées seront liées uniquement à ce projet.

Introduction au gestionnaire de paquets pip :

Python utilise un gestionnaire de paquets appelé pip. Cet outil permet d'installer facilement des bibliothèques Python depuis Internet afin d'ajouter des fonctionnalités supplémentaires à un projet.

Le principe est simple : il suffit d'utiliser une commande dans le terminal pour télécharger et installer une bibliothèque ainsi que toutes ses dépendances. La commande générale est la suivante :

pip install nom_du_package

Dans ce TP, pip sera utilisé pour installer le framework Django qui sera nécessaire pour la création de l'application web.

Installation de Django :

Après avoir vérifié l'installation de Python et créé un environnement virtuel, l'étape suivante consiste à installer le framework Django. Pour cela, on utilise le gestionnaire de paquets pip dans le terminal avec la commande suivante :

pip install django

Cette commande permet de télécharger Django ainsi que les dépendances nécessaires à son fonctionnement. Une fois l'installation terminée, le terminal affiche un message indiquant que l'installation a réussi.

Pour vérifier que Django est correctement installé, on utilise la commande suivante

django-admin --version

Si l'installation s'est bien déroulée, le terminal affiche le numéro de version de Django installé, par exemple 4.x.x. Cela confirme que Django est prêt à être utilisé pour la création du projet.

```
(env) PS C:\Users\DELL\Desktop\Tp_3> pip install django
Collecting django
  Downloading django-6.0.3-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from django)
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from django)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from django)
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-6.0.3-py3-none-any.whl (8.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.4/8.4 MB 954.4 kB/s  0:00:08
Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.3 sqlparse-0.5.5 tzdata-2025.3

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(env) PS C:\Users\DELL\Desktop\Tp_3> django-admin --version
6.0.3
```

Création d'un projet Django :

Une fois Django installé, il est possible de créer un nouveau projet. Dans Django, un projet représente l'ensemble d'un site web et contient la configuration générale. Un projet peut également contenir plusieurs applications, chaque application ayant une fonction spécifique comme un blog ou un système de gestion d'utilisateurs.

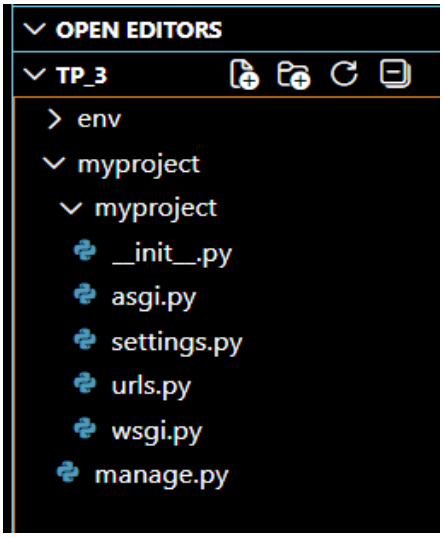
Pour créer un projet Django, on utilise la commande suivante dans le terminal :

django-admin startproject myproject

```
(env) PS C:\Users\DELL\Desktop\Tp_3> django-admin startproject myproject
(env) PS C:\Users\DELL\Desktop\Tp_3>
```

Cette commande crée un nouveau projet nommé myproject avec tous les fichiers nécessaires au fonctionnement de base du framework.

Après la création du projet, plusieurs fichiers importants sont générés automatiquement.



Le fichier manage.py permet d'exécuter différentes commandes Django à partir du terminal. Par exemple, il permet de lancer le serveur de développement, d'appliquer les migrations de la base de données ou encore de créer une nouvelle application.

Le fichier settings.py contient la configuration principale du projet. Il permet de définir les applications installées, la configuration de la base de données, les templates HTML ainsi que les fichiers statiques utilisés par le projet.

Le fichier urls.py est utilisé pour définir les différentes routes URL du site web. Chaque URL est associée à une fonction Python qui détermine la page ou la réponse qui sera affichée.

Le fichier wsgi.py permet la communication entre l'application Django et le serveur web utilisé pour exécuter le projet.

Le fichier asgi.py est une version plus moderne permettant de gérer des applications asynchrones.

Ensuite, il faut se déplacer dans le dossier du projet avec la commande suivante :

```
Cd myproject
```

Création d'une application dans le projet Django :

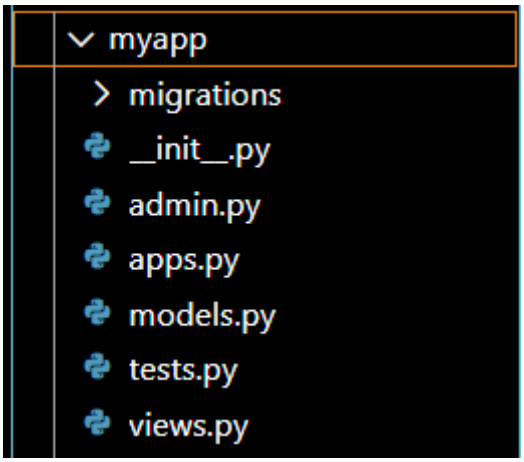
Après la création du projet Django, l'étape suivante consiste à créer une application. Dans Django, une application représente un module qui réalise une fonctionnalité spécifique à l'intérieur du projet.

Pour créer une application appelée myapp, on utilise la commande suivante dans le terminal :

```
python manage.py startapp myapp
```

```
(env) PS C:\Users\DELL\Desktop\Tp_3> cd myproject
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject> python manage.py startapp myapp
```

Cette commande génère automatiquement un dossier nommé myapp contenant plusieurs fichiers nécessaires au développement de l'application, comme les fichiers pour les vues, les modèles et les configurations.



Exécution du serveur Django :

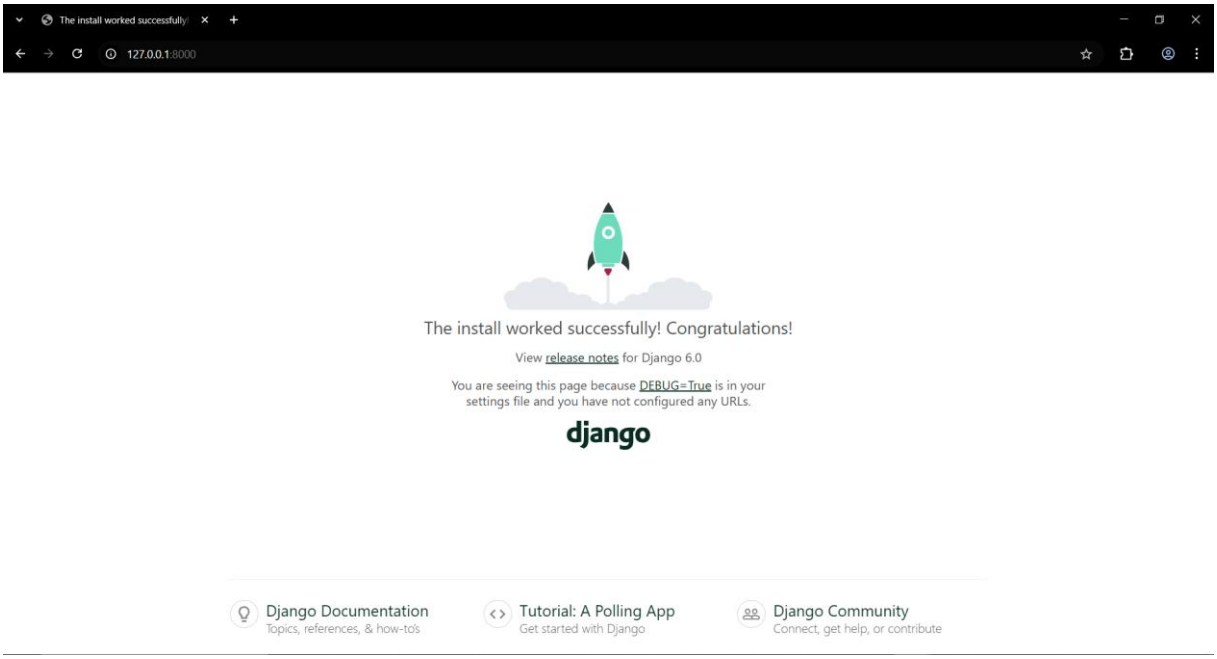
Pour lancer le serveur de développement, il faut d'abord accéder au dossier contenant le fichier manage.py. Ensuite, on exécute la commande suivante dans le terminal :

python manage.py runserver

```
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
March 13, 2026 - 17:10:28
Django version 6.0.3, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[13/Mar/2026 17:13:25] "GET / HTTP/1.1" 200 12068
Not Found: /favicon.ico
[13/Mar/2026 17:13:25] "GET /favicon.ico HTTP/1.1" 404 2211
```

Après l'exécution de cette commande, le terminal affiche un message indiquant que le serveur a démarré, avec une adresse comme http://127.0.0.1:8000/.

En ouvrant cette adresse dans un navigateur web, la page d'accueil par défaut de Django apparaît, ce qui confirme que le projet fonctionne correctement.



Vues, URLs et Templates : Afficher la Première Page

- Configuration des URLs et des Vues :

Pour commencer à afficher une page dans notre application Django, il faut relier une URL à une vue. Une vue est une fonction Python qui reçoit la requête de l'utilisateur et renvoie une réponse.

Dans l'application myapp, un fichier urls.py a été créé afin de gérer les URLs propres à cette application. Dans ce fichier, nous avons importé la fonction path ainsi que le fichier views. Ensuite, nous avons défini une route qui associe l'URL principale de l'application à la vue index.

```
myproject > myapp > urls.py > ...
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
]
```

Après cela, il faut connecter les URLs de l'application au projet principal. Pour cela, le fichier myproject/urls.py a été modifié afin d'inclure les URLs de myapp à l'aide de la fonction include. Ainsi, toutes les requêtes envoyées à l'URL principale du site seront redirigées vers les routes définies dans l'application.

```
myproject > myproject > urls.py > ...
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls'))
]
```

- Utilisation des Templates :

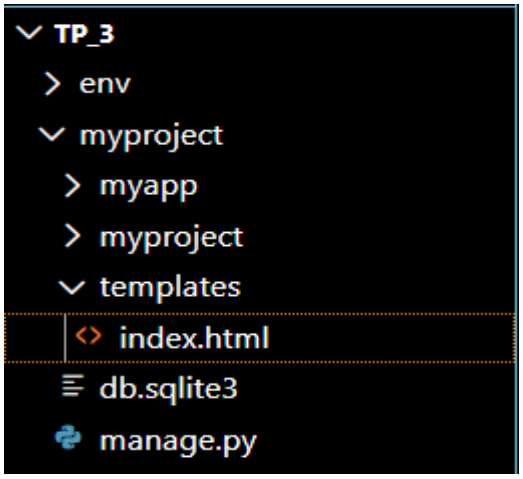
Écrire directement du HTML dans les vues n'est pas une bonne pratique dans Django. Pour séparer la logique de l'application de l'interface utilisateur, Django utilise un système de templates qui permet de générer les pages HTML de manière plus organisée.

Pour commencer, un dossier nommé templates a été créé à la racine du projet, au même niveau que le fichier manage.py. Ce dossier servira à stocker les fichiers HTML utilisés par l'application.

Ensuite, il est nécessaire de configurer Django afin qu'il puisse localiser ce dossier. Pour cela, le fichier myproject/settings.py a été modifié au niveau de la variable TEMPLATES. La clé DIRS a été configurée pour inclure le dossier templates du projet.

```
myproject > myproject > settings.py > ...
56  TEMPLATES = [
57      {
58          'BACKEND': 'django.template.backends.django.DjangoTemplates',
59          'DIRS': [BASE_DIR, 'templates'],
60          'APP_DIRS': True,
61          'OPTIONS': {
62              'context_processors': [
63                  'django.template.context_processors.request',
64                  'django.contrib.auth.context_processors.auth',
65                  'django.contrib.messages.context_processors.messages',
66              ],
67          },
68      },
69  ]
70
```

Après cette configuration, un premier fichier template nommé index.html a été créé dans le dossier templates. Ce fichier contiendra le code HTML qui sera affiché dans le navigateur lorsque la vue correspondante sera appelée.



- Passer des données dynamiques au Template :

L'un des avantages du moteur de templates de Django est la possibilité d'afficher des données dynamiques dans les pages HTML. Pour cela, la vue peut envoyer des informations au template à l'aide d'un dictionnaire appelé context.

Dans le fichier myapp/views.py, la vue index a été modifiée afin de créer un dictionnaire contenant différentes données comme le nom, l'âge et une valeur booléenne. Ce dictionnaire est ensuite envoyé au template grâce à la fonction render.

```
myproject > myapp > views.py > ...
1   from django.shortcuts import render
2   from django.http import HttpResponse
3
4   # Create your views here.
5   def index(request):
6       context = {
7           'name': 'aya',
8           'age' : 21 ,
9           'morrocan' : True
10      }
11      return render(request, 'index.html' , context)
```

Dans le fichier templates/index.html, ces données peuvent être affichées en utilisant la syntaxe des templates Django avec des doubles accolades {{ }}. Cette syntaxe permet d'insérer dynamiquement les valeurs envoyées par la vue.

```
myproject > templates > index.html > h1
1   <h1> Test index </h1>
2   <h1> Name : {{name}} </h1>
3   <h1> Age : {{age}} </h1>
```

Lorsque l'utilisateur accède à l'URL du site, la vue envoie les données au template qui les affiche dans la page HTML. Le résultat final apparaît alors dans le navigateur, comme montré dans la capture d'écran.

Gestion des formulaires : création d'un compteur de mots

Les formulaires permettent aux utilisateurs d'interagir avec une application web en envoyant des données au serveur. Dans cette partie, nous avons créé un formulaire simple permettant de compter le nombre de mots contenus dans un texte saisi par l'utilisateur.

Tout d'abord, un formulaire HTML a été ajouté dans le fichier templates/index.html. Ce formulaire contient une zone de texte dans laquelle l'utilisateur peut écrire un message, ainsi qu'un bouton permettant d'envoyer les données.

```
myproject > templates > index.html > ...
1  <h1> Input text below</h1>
2  <form method="" action="counter">
3      <textarea name="text" rows="25" cols="25" id=""></textarea><br>
4      <input type="submit" value="compter">
5  </form>
```

Ensuite, une nouvelle vue appelée counter a été créée dans le fichier views.py. Cette vue récupère le texte envoyé par le formulaire à l'aide de request.GET. Le texte est ensuite séparé en mots à l'aide de la fonction split(), puis le nombre de mots est calculé avec la fonction len().

```
myproject > myapp > views.py > counter
13  def counter(request):
14      text = request.GET['text']
15      amount_of_words = len(text.split())
16
17      return render(request, 'counter.html', {'amount': amount_of_words })
```

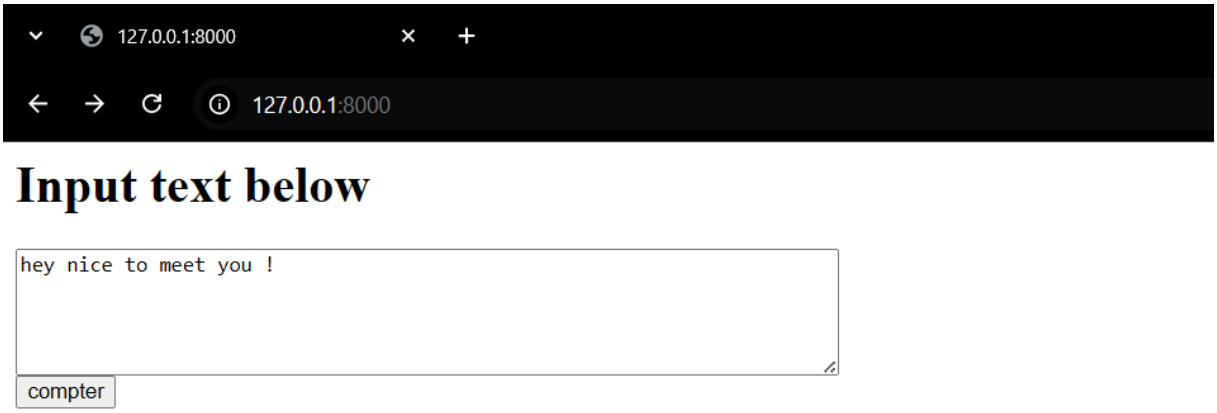
Après cela, une nouvelle route a été ajoutée dans le fichier myapp/urls.py afin de relier l'URL /counter à la vue counter.

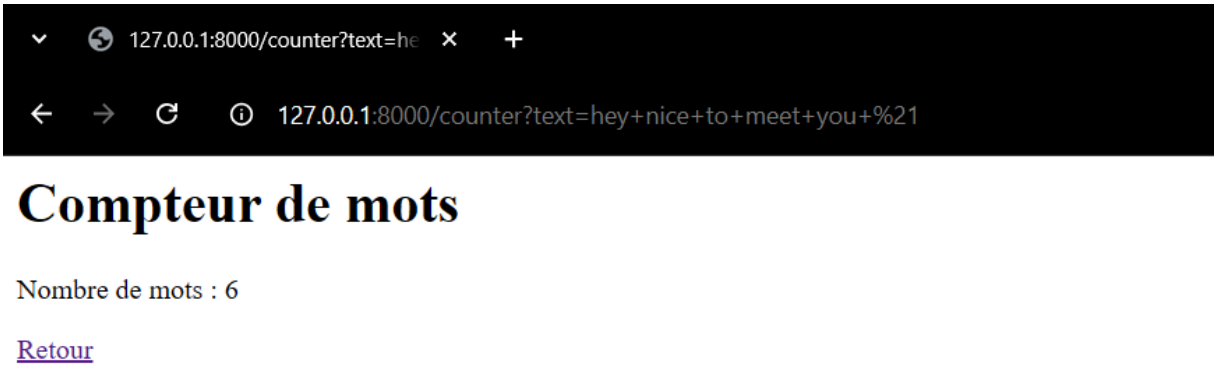
```
myproject > myapp > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.index, name="index"),
6      path('counter', views.counter, name="counter")
7  ]
```

Un nouveau template nommé counter.html a également été créé dans le dossier templates. Ce fichier sert à afficher le résultat du compteur de mots en utilisant la variable amount envoyée par la vue.

```
myproject > templates > counter.html > p
1  <h1>Compteur de mots </h1>
2  <p>Nombre de mots : {{amount}}</p>
3
```

Enfin, l'application a été testée en lançant le serveur Django. En accédant à la page d'accueil, l'utilisateur peut entrer un texte dans le formulaire et cliquer sur le bouton. Le système redirige ensuite vers la page de résultat qui affiche le nombre de mots calculé. Les captures d'écran ci-dessous illustrent les différentes étapes ainsi que le résultat obtenu.





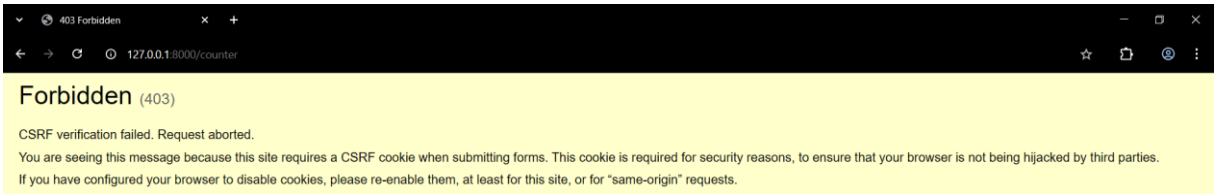
- Tester le formulaire POST :

Après la création du formulaire et de la vue du compteur de mots, l'étape suivante consiste à tester l'envoi des données en utilisant la méthode POST.

Dans un premier temps, le formulaire dans le fichier index.html a été modifié pour utiliser la méthode POST au lieu de GET.

```
myproject > templates > index.html > html > head
18  <body>
19  <h1> Input text below</h1>
20  <form method="POST" action="counter">
21
22      <textarea name="text" rows="25" cols="25" id=""></textarea><br>
23      <input type="submit" value="compter">
24  </form>
25
26  </body>
27  </html>
```

Cependant, lors du premier test, une erreur 403 Forbidden est apparue. Cette erreur est due au système de sécurité de Django qui exige l'utilisation d'un jeton CSRF pour les formulaires envoyés avec la méthode POST.



Pour corriger ce problème, la balise csrf_token a été ajoutée à l'intérieur du formulaire. Ce jeton permet à Django de vérifier que la requête provient bien du site et non d'une source externe.

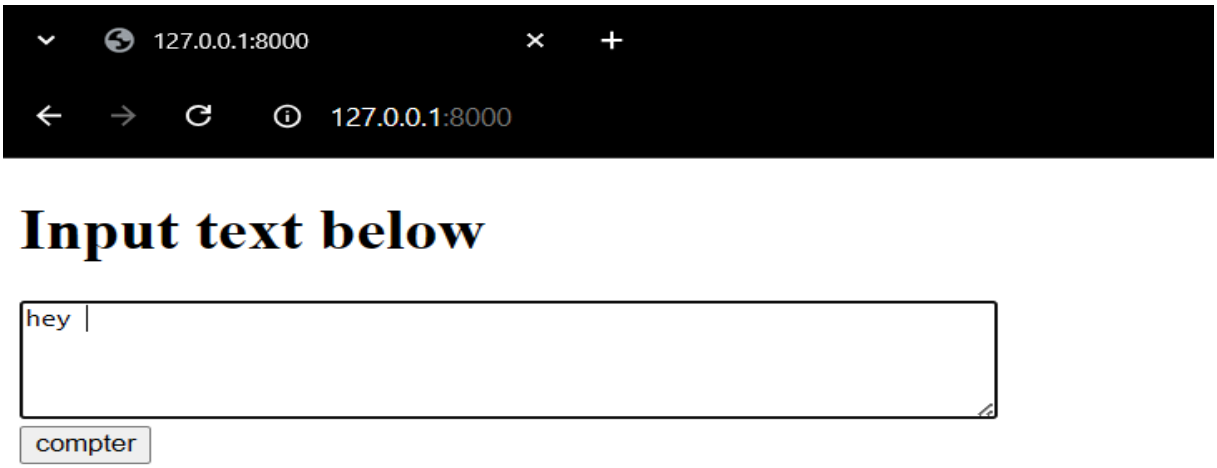
```
myproject > templates > index.html > html
20  <form method="POST" action="counter">
21      {% csrf_token %}
22      <textarea name="text" rows="25" cols="25" id=""></textarea><br>
23      <input type="submit" value="compter">
24  </form>
```

Après cette correction, une autre erreur est apparue car la vue récupérait encore les données avec request.GET.

Comme le formulaire utilise maintenant la méthode POST, il a fallu modifier le code dans views.py pour utiliser request.POST afin de récupérer correctement le texte envoyé par l'utilisateur.

```
myproject > myapp > views.py > counter
12
13  def counter(request):
14      text = request.POST['text']
15      amount_of_words = len(text.split())
16
17      return render(request, 'counter.html', {'amount': amount_of_words
```

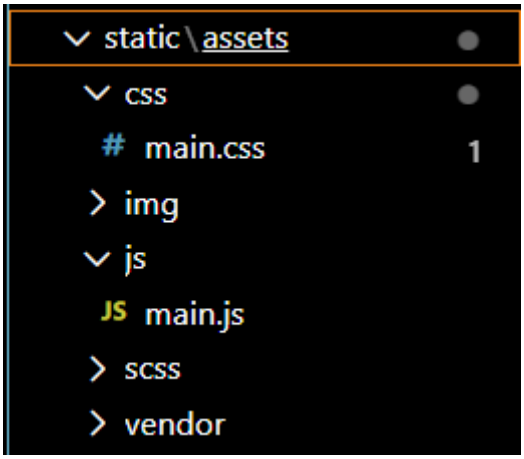
Une fois ces modifications effectuées, le formulaire a été testé à nouveau. L'utilisateur peut entrer du texte dans la zone prévue, cliquer sur le bouton, et le système affiche ensuite la page contenant le nombre de mots calculé.



Intégration des Fichiers Statiques (CSS/JS du template Bootstrap) :

Dans une application web, les fichiers statiques correspondent aux fichiers qui ne changent pas, comme les fichiers CSS, JavaScript ou les images. Dans cette partie, un template Bootstrap a été intégré afin d'améliorer l'apparence du site.

Tout d'abord, le template OnePage a été téléchargé puis décompressé. Ensuite, un dossier nommé static a été créé à la racine du projet Django, au même niveau que le fichier manage.py. Les fichiers du template, notamment le dossier assets contenant les fichiers CSS, JavaScript et les images, ont été copiés dans ce dossier static.



Après cela, le fichier settings.py du projet a été modifié pour configurer l'utilisation des fichiers statiques.

```
myproject > myproject > settings.py > ...
from pathlib import Path
import os

```

La variable STATICFILES_DIRS a été ajoutée afin d'indiquer à Django l'emplacement du dossier static.

```
myproject > myproject > settings.py > ...
STATIC_URL = 'static/'
STATICFILES_DIRS = (os.path.join(BASE_DIR, 'static'),)

```

Ensuite, le contenu du fichier index.html du template téléchargé a été copié dans le fichier templates/index.html du projet.

Les chemins vers les fichiers CSS, JavaScript et les images ont également été modifiés afin d'utiliser la balise {% static %}. Cela permet à Django de générer correctement les liens vers les fichiers situés dans le dossier static.

```
51  <!-- Vendor CSS Files -->
52  <link href="{% static 'assets/vendor/bootstrap/css/bootstrap.min.css' %}" rel="stylesheet">
53  <link href="{% static 'assets/vendor/bootstrap-icons/bootstrap-icons.css' %}" rel="stylesheet">
54  <link href="{% static 'assets/vendor/aos/aos.css' %}" rel="stylesheet">
55  <link href="{% static 'assets/vendor/glightbox/css/glightbox.min.css' %}" rel="stylesheet">
56  <link href="{% static 'assets/vendor/swiper/swiper-bundle.min.css' %}" rel="stylesheet">
57
58  <!-- Main CSS File -->
59  <link href="{% static 'assets/css/main.css' %}" rel="stylesheet">
60
```

```
284          <div class="col-xl-2 col-md-3 col-6 client-logo">
285            <img src="{% static 'assets/img/clients/client-1.png' %}" class="img-fluid" alt="">
286          </div><!-- End Client Item -->
287
288          <div class="col-xl-2 col-md-3 col-6 client-logo">
289            <img src="{% static 'assets/img/clients/client-2.png' %}" class="img-fluid" alt="">
290          </div><!-- End Client Item -->
291
292          <div class="col-xl-2 col-md-3 col-6 client-logo">
293            <img src="{% static 'assets/img/clients/client-3.png' %}" class="img-fluid" alt="">
294          </div><!-- End Client Item -->
295
296          <div class="col-xl-2 col-md-3 col-6 client-logo">
297            <img src="{% static 'assets/img/clients/client-4.png' %}" class="img-fluid" alt="">
298          </div><!-- End Client Item -->
299
300          <div class="col-xl-2 col-md-3 col-6 client-logo">
301            <img src="{% static 'assets/img/clients/client-5.png' %}" class="img-fluid" alt="">
302          </div><!-- End Client Item -->
303
304          <div class="col-xl-2 col-md-3 col-6 client-logo">
305            <img src="{% static 'assets/img/clients/client-6.png' %}" class="img-fluid" alt="">
306          </div><!-- End Client Item -->
```

```
360        <img src="{% static 'assets/img/testimonials/testimonials-1.jpg' %}" class="testimonial-img" alt="">
361        <h3>Saul Goodman</h3>
362        <h4>Ceo &amp; Founder</h4>
```

```
1059  <!-- Vendor JS Files -->
1060  <script src="{% static 'assets/vendor/bootstrap/js/bootstrap.bundle.min.js' %}"></script>
1061  <script src="{% static 'assets/vendor/php-email-form/validate.js' %}"></script>
1062  <script src="{% static 'assets/vendor/aos/aos.js' %}"></script>
1063  <script src="{% static 'assets/vendor/purecounter/purecounter_vanilla.js' %}"></script>
1064  <script src="{% static 'assets/vendor/glightbox/js/glightbox.min.js' %}"></script>
1065  <script src="{% static 'assets/vendor/swiper/swiper-bundle.min.js' %}"></script>
1066  <script src="{% static 'assets/vendor/imagesloaded/imagesloaded.pkgd.min.js' %}"></script>
1067  <script src="{% static 'assets/vendor/isotope-layout/isotope.pkgd.min.js' %}"></script>
1068
1069  <!-- Main JS File -->
1070  <script src="{% static 'assets/js/main.js' %}"></script>
```

Lors du premier test, une erreur de type TemplateSyntaxError est apparue car la balise static n'avait pas encore été chargée.

TemplateSyntaxError at / 127.0.0.1:8000

TemplateSyntaxError at /

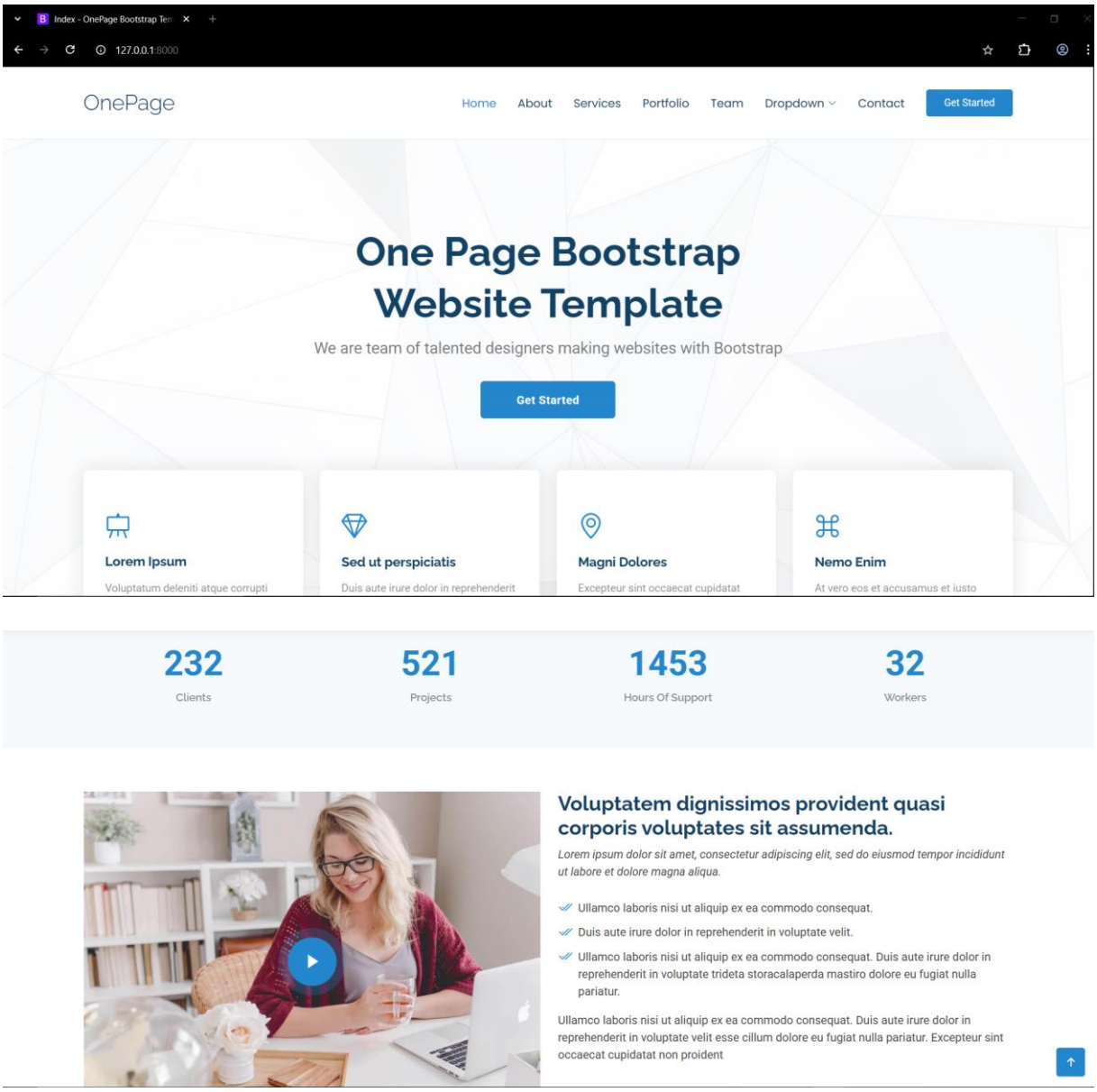
Invalid block tag on line 7: 'static'. Did you forget to register or load this tag?

Request Method:	GET
Request URL:	http://127.0.0.1:8000/
Django Version:	6.0.3
Exception Type:	TemplateSyntaxError
Exception Value:	Invalid block tag on line 7: 'static'. Did you forget to register or load this tag?
Exception Location:	C:\Users\DELL\Desktop\Tp_3\env\Lib\site-packages\django\template\base.py, line 644, in invalid_block_tag
Raised during:	myapp.views.index
Python Executable:	C:\Users\DELL\Desktop\Tp_3\env\Scripts\python.exe
Python Version:	3.14.3
Python Path:	['C:\\Users\\DELL\\Desktop\\Tp_3\\myproject', 'C:\\Users\\DELL\\AppData\\Local\\Programs\\Python\\Python314\\python314.zip', 'C:\\Users\\DELL\\AppData\\Local\\Programs\\Python\\Python314\\DLLs', 'C:\\Users\\DELL\\AppData\\Local\\Programs\\Python\\Python314\\Lib', 'C:\\Users\\DELL\\AppData\\Local\\Programs\\Python\\Python314', 'C:\\Users\\DELL\\Desktop\\Tp_3\\env', 'C:\\Users\\DELL\\Desktop\\Tp_3\\env\\Lib\\site-packages']
Server time:	Sat, 14 Mar 2026 14:41:53 +0000

Pour permettre à Django d'accéder aux fichiers statiques, la balise {% load static %} a été ajoutée au début du fichier.

```
myproject > templates > index.html > ...
32  {% load static %}
33  <!DOCTYPE html>
34  <html lang="en">
```

Après l'ajout de la balise ,le problème a été corrigé et les fichiers CSS et JavaScript ont été correctement chargés dans le projet.



Modèles de Données et Migrations

Dans Django, les modèles représentent la structure des données de l'application sous forme de classes Python. Grâce à l'ORM (Object Relational Mapper) de Django, ces classes sont automatiquement transformées en tables dans la base de données.

- Définition du modèle :

Dans le fichier myapp/models.py, un modèle nommé Feature a été créé pour représenter une fonctionnalité du site. Ce modèle contient deux champs de type texte :

- name : le nom de la fonctionnalité
- details : une description de la fonctionnalité

```
myproject > myapp > models.py > ...
1  from django.db import models
2
3  # Create your models here.
4
5  class Feature(models.Model) :
6      name = models.CharField(max_length=100)
7      details = models.CharField(max_length=100)
```

Chaque attribut du modèle correspond à une colonne dans la table de la base de données.

- Enregistrement de l'application :

Pour que Django puisse détecter les modèles de l'application, celle-ci doit être déclarée dans la liste INSTALLED_APPS du fichier myproject/settings.py.

Dans notre cas, l'application myapp a été ajoutée :

```
myproject > myproject > settings.py > ...
32  # Application definition
33
34  INSTALLED_APPS = [
35      'django.contrib.admin',
36      'django.contrib.auth',
37      'django.contrib.contenttypes',
38      'django.contrib.sessions',
39      'django.contrib.messages',
40      'django.contrib.staticfiles',
41      'myapp',
42  ]
```

Cela permet à Django de prendre en compte les modèles définis dans cette application.

- Création des migrations :

Les migrations sont des fichiers qui décrivent les modifications à appliquer à la base de données lorsque des modèles sont créés ou modifiés.

Pour générer les migrations, la commande suivante a été exécutée :

python manage.py makemigrations

Cette commande crée un fichier de migration dans le dossier migrations de l'application.

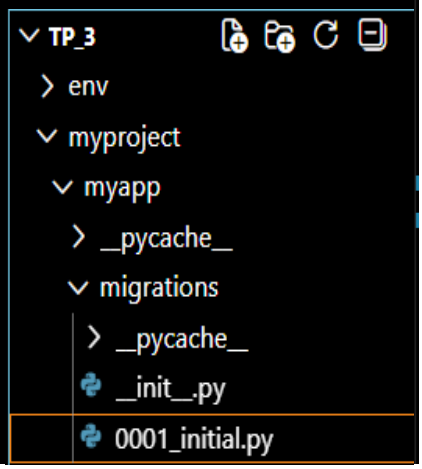
- Application des migrations :

Ensuite, les migrations ont été appliquées à la base de données avec la commande

python manage.py migrate

Django a alors créé les tables correspondantes dans la base de données pour toutes les applications, y compris le modèle Feature de myapp.

```
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject> python manage.py makemigrations
Migrations for 'myapp':
  myapp\migrations\0001_initial.py
    + Create model Feature
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, myapp, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying myapp.0001_initial... OK
  Applying sessions.0001_initial... OK
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject>
```



L'Interface d'Administration de Django

Django fournit une interface d'administration intégrée qui permet de gérer facilement les données stockées dans la base de données. Cette interface permet d'ajouter, modifier ou supprimer des objets sans avoir besoin de créer une interface spécifique dans le site.

- Création d'un super-utilisateur :

Pour accéder à l'interface d'administration, il est nécessaire de créer un super-utilisateur. Cette opération se fait à l'aide de la commande suivante :

python manage.py createsuperuser

Lors de l'exécution de cette commande, Django demande de saisir :

- le nom d'utilisateur
- l'adresse email
- le mot de passe

Une fois ces informations saisies, le super-utilisateur est créé avec succès.

```
PS C:\Users\DELL\Desktop\Tp_3> & c:\Users\DELL\Desktop\Tp_3\env\Scripts\Activate.ps1
(env) PS C:\Users\DELL\Desktop\Tp_3> cd myproject
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject> python manage.py createsuperuser
Username (leave blank to use 'dell'): admin
Email address: admin@gmail.com
Password:
Password (again):
Superuser created successfully.
(env) PS C:\Users\DELL\Desktop\Tp_3\myproject>
```

- Enregistrement du modèle dans l'admin :

Pour que le modèle Feature apparaisse dans l'interface d'administration, il doit être enregistré dans le fichier : myapp/admin.py

```
myproject > myapp > admin.py
1  from django.contrib import admin
2  from .models import Feature
3
4  # Register your models here.
5
6  admin.site.register(Feature)
```

Cette étape permet à Django d'afficher et de gérer les objets du modèle Feature dans l'interface d'administration.

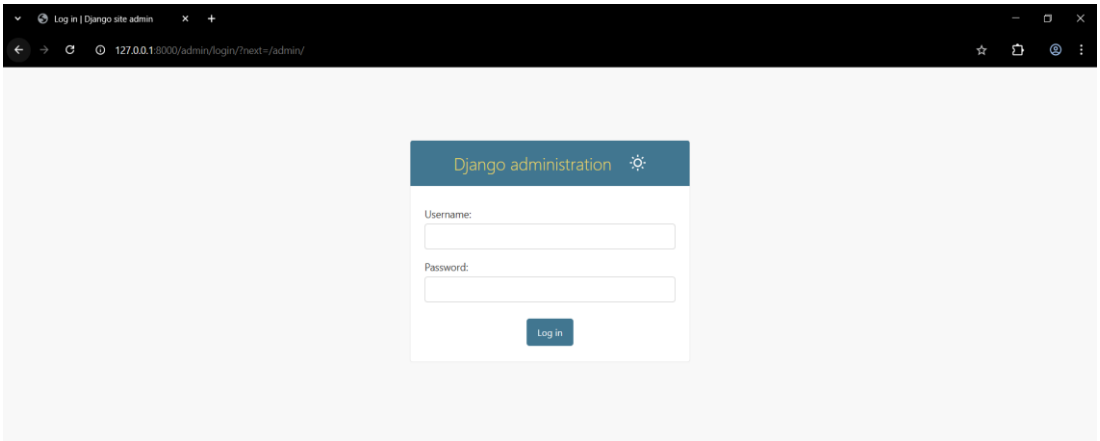
- Accès à l'interface d'administration :

Après avoir lancé le serveur avec la commande : python manage.py runserver

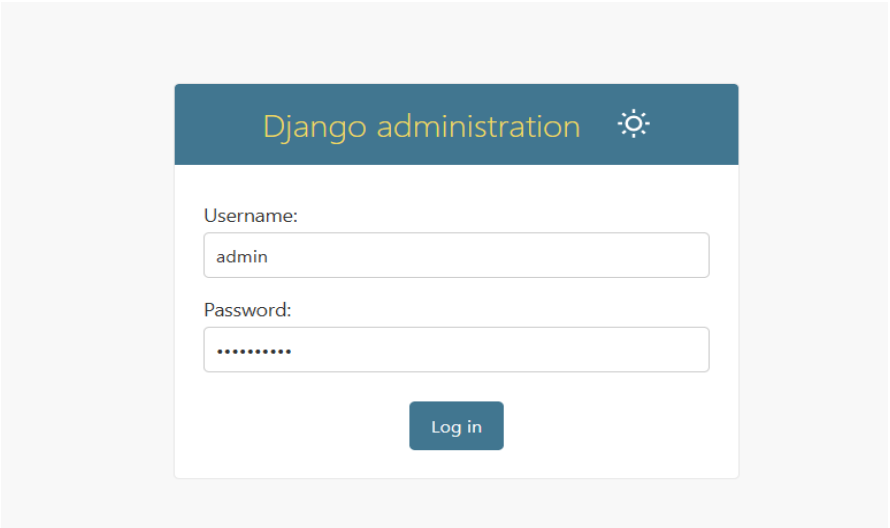
L'interface d'administration est accessible à l'adresse :

http://127.0.0.1:8000/admin/

Une page de connexion apparaît.



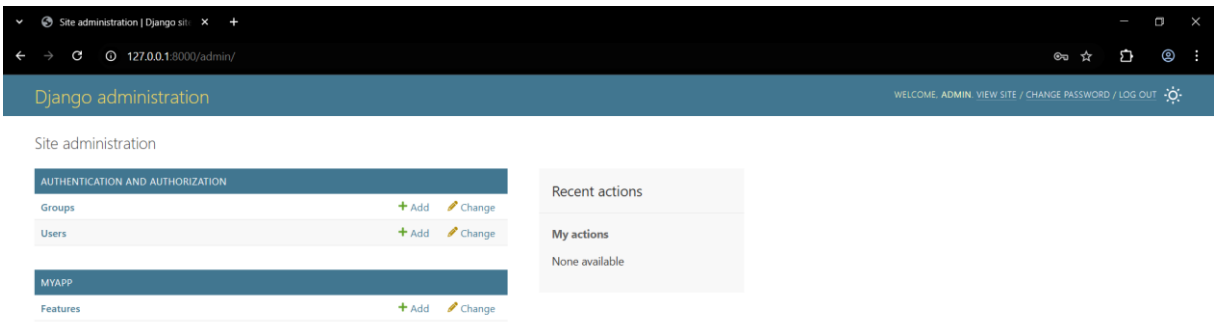
L'utilisateur doit saisir le nom d'utilisateur et le mot de passe du super-utilisateur créé précédemment.



Après la connexion, le tableau de bord de l'administration Django s'affiche.

Gestion des données du modèle :

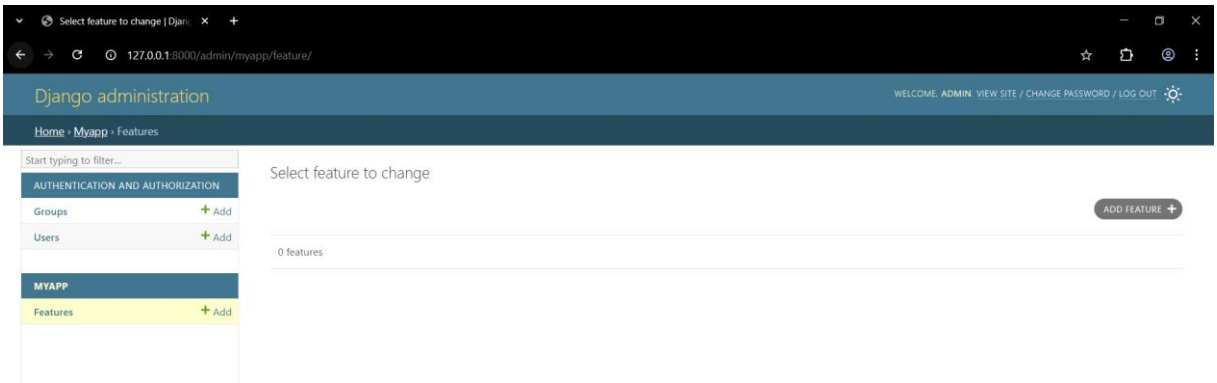
Dans l'interface d'administration, une section MYAPP apparaît avec le modèle Features.

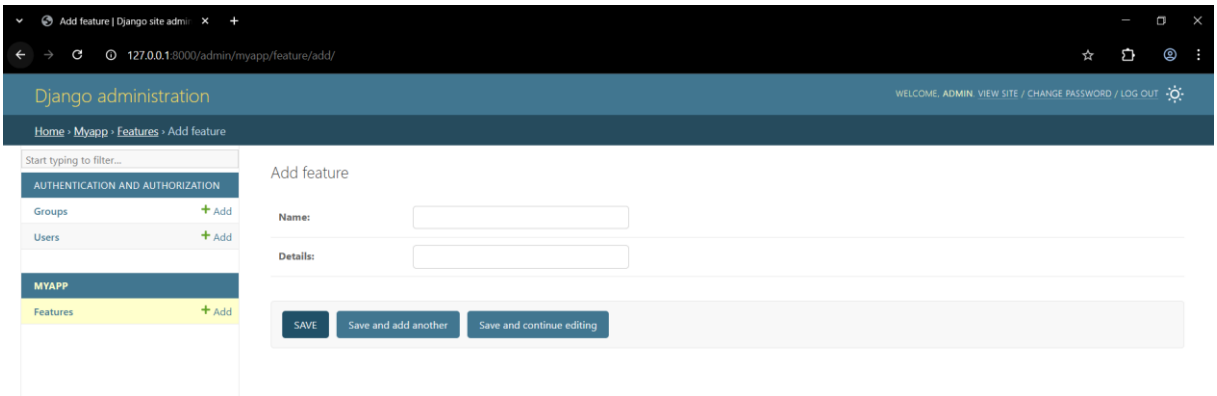


En cliquant sur Features, il est possible de :

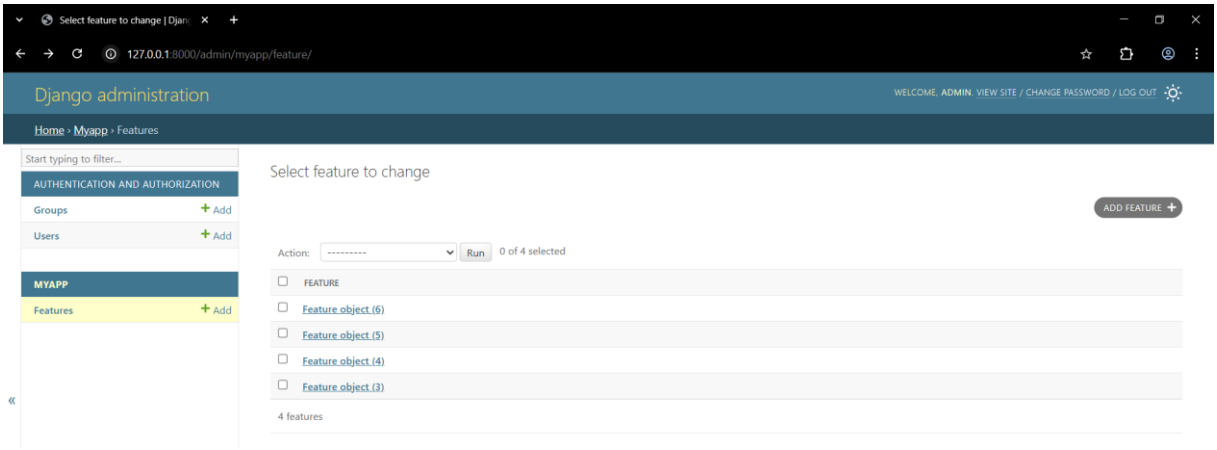
- consulter les objets existants
- ajouter de nouvelles fonctionnalités
- modifier ou supprimer des fonctionnalités

Pour ajouter une nouvelle fonctionnalité, il suffit de cliquer sur le bouton Add Feature, puis de remplir les champs Name et Details.





Plusieurs fonctionnalités ont été ajoutées via cette interface afin de les afficher ensuite dans le site.



- Affichage des données dans le template :

Afin d'afficher les données stockées dans la base de données, la vue index a été modifiée dans le fichier views.py.

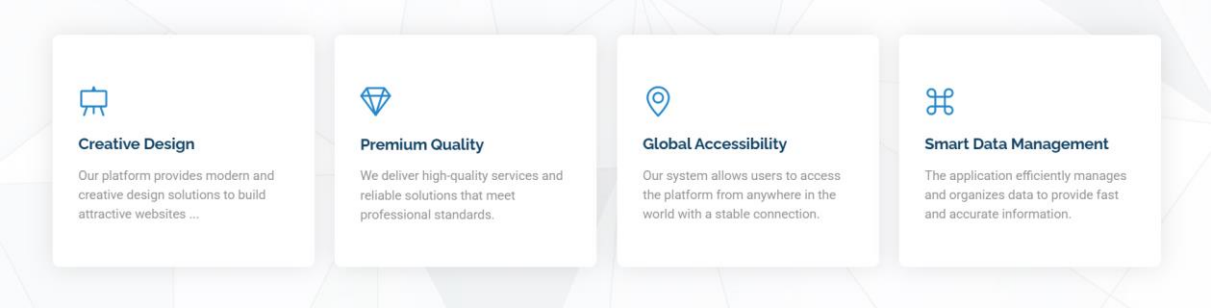
La vue récupère toutes les fonctionnalités enregistrées dans la base de données :

```
myproject > myapp > views.py > counter
def index(request):
    features = Feature.objects.all()
    return render(request, 'index.html' , {'features' : features})
```

Ensuite, dans le template index.html, une boucle Django Template est utilisée pour parcourir la liste des fonctionnalités et les afficher dynamiquement :

```
myproject > templates > index.html > html > body.index-page > main.main
 33  <html lang="en">
 70  <body class="index-page">
115    <main class="main">
118      <section id="hero" class="hero section">
122        <div class="container">
132
133          <div class = "row gy-4 mt-5">
134           {% for feature in features %}
135
136           <div class = "col-md-6 col-lg-3" data-aos="zoom-out">
137             <div class="icon-box">
138               {% if forloop.counter == 1 %}
139               <div class="icon"><i class="bi bi-easel"></i></div>
140               {% elif forloop.counter == 2 %}
141               <div class="icon"><i class="bi bi-gem"></i></div>
142               {% elif forloop.counter == 3 %}
143               <div class="icon"><i class="bi bi-geo-alt"></i></div>
144               {% else %}
145               <div class="icon"><i class="bi bi-command"></i></div>
146               {% endif %}
147               <h4 class="title"><a href="">{{ feature.name }}</a></h4>
148               <p class="description">{{ feature.details }}</p>
149             </div>
150           </div>
151           {% endfor %}
152         </div>
```

Grâce à cette modification, les fonctionnalités ajoutées depuis l'interface d'administration apparaissent automatiquement sur la page principale du site.



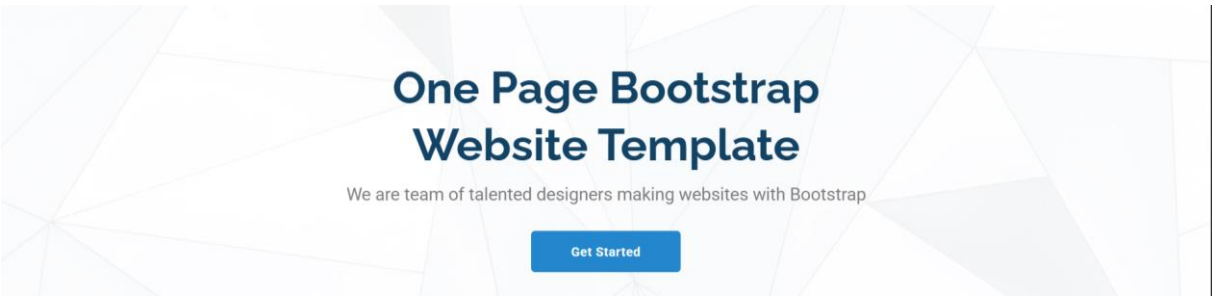
Authentification des Utilisateurs

Django fournit un système d'authentification intégré permettant de gérer facilement les utilisateurs d'une application web.
Dans ce projet, nous avons implémenté trois fonctionnalités principales :

- l'inscription (Register)
- la connexion (Login)
- la déconnexion (Logout)

1. Inscription des utilisateurs (Register) :

L'inscription permet à un nouvel utilisateur de créer un compte afin d'accéder aux fonctionnalités du site.



Création du template register.html :

Un template register.html a été créé dans le dossier templates.
Ce formulaire permet à l'utilisateur de saisir les informations nécessaires :

- Username
- Email
- Password
- Confirm password

Le formulaire utilise la méthode POST et inclut le csrf_token pour la sécurité.

```
myproject > templates > register.html > html > body.login > div#container > div#content > div#content-main > form#login-form > div.form-row > input#id_password
1   {% load static %}
2   <!DOCTYPE html>
3   <html lang="en">
4   <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <link rel="stylesheet" href="{% static 'admin/css/base.css' %}">
8     <link rel="stylesheet" href="{% static 'admin/css/login.css' %}">
9     <link rel="stylesheet" href="{% static 'admin/css/responsive.css' %}">
10    <title>Register</title>
11  </head>
12
13  <body class="login">
14    <div id="container">
15      <div id="header">
16        <h1>Register</h1>
17      </div>
18      <div id="content">
19        <div id="content-main">
20          <form action="register" method="post" id="login-form">
21
22          {% csrf_token %}
23
24          <div class="form-row">
25          <label for="id_username">Username :</label>
26          <input type="text" name="username" id="id_username">
27          </div>
29          <div class="form-row">
30          <label for="id_email">Email :</label>
31          <input type="email" name="email" id="id_username">
32          </div>
33
34          <div class="form-row">
35          <label for="id_password">Password :</label>
36          <input type="password" name="password" id="id_password">
37          </div>
38
39          <div class="form-row">
40          <label for="id_password2">Confirm password :</label>
41          <input type="password" name="password2" id="id_password">
42          </div>
43
44          <div class="submit-row">
45          <input type="submit" value="Register" class="default">
46          </div>
47          </form>
48  </body>
49  </html>
```

Une fois rempli, l'utilisateur clique sur Register pour envoyer les données au serveur.

- Configuration de l'URL :

L'URL de la page d'inscription est définie dans le fichier urls.py :

```
myproject > myapp > urls.py > ...
8
9   path('register' , views.register , name= "register"),
10
```

Cette URL appelle la fonction register définie dans views.py.

- Fonction register dans views.py :

La fonction register récupère les données envoyées par le formulaire et crée un nouvel utilisateur.

```
1  from django.shortcuts import render , redirect
2  from django.http import HttpResponse
3  from django.contrib.auth.models import User , auth
4  from django.contrib import messages
5  from .models import Feature
```

```
myproject > myapp > views.py > login
20  def register(request):
21      if request.method == 'POST':
22          username = request.POST['username']
23          email  = request.POST['email']
24          password = request.POST['password']
25          password2 = request.POST['password2']
26
27          if password == password2 :
28              if User.objects.filter(email=email).exists():
29                  messages.info(request , 'email exists déjà')
30                  return redirect('register')
31              elif User.objects.filter(username=username).exists():
32                  messages.info(request, 'username existe déjà')
33                  return redirect('register')
34              else :
35                  user = User.objects.create_user(username=username,email=email,password=password)
36                  user.save()
37                  return redirect('login')
38          else:
39              messages.info(request,'MDP non identiques')
40              return redirect('register')
41      else:
42          return render(request,'register.html')
```

Cette fonction vérifie :

- si les mots de passe correspondent
- si l'email existe déjà
- si le nom d'utilisateur existe déjà

Si toutes les conditions sont respectées, l'utilisateur est créé dans la base de données.

- Affichage conditionnel du bouton Register :

Dans le fichier index.html, une condition a été ajoutée pour afficher le bouton Register uniquement lorsque l'utilisateur n'est pas authentifié.

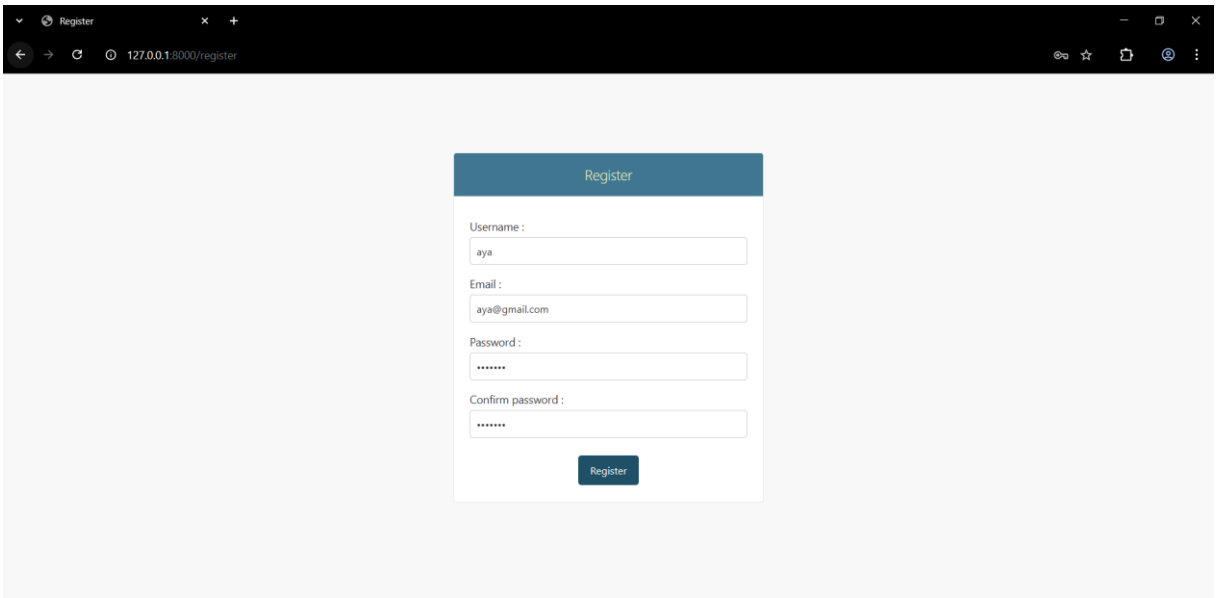
Cette condition utilise la variable user.is_authenticated fournie par le système d'authentification de Django.

```
myproject > templates > index.html > html > body.index-page > main.main > section#hero.hero.section > div.container > div.row.gy-4.mt-5
 33  <html lang="en">
 70  <body class="index-page">
119    <main class="main">
122      <section id="hero" class="hero section">
126        <div class="container">
134          <div class="text-center" data-aos="zoom-out" data-aos-delay="100">
135
136            {% if not user.is_authenticated %}
137            <a href="register" class="btn-get-started">Register</a>
138            {% endif %}
139
140          </div>
141
```

Explication

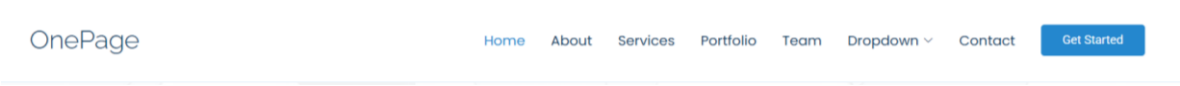
- user.is_authenticated : vérifie si l'utilisateur est connecté.
- not user.is_authenticated : signifie que l'utilisateur n'est pas connecté.
- Si l'utilisateur n'est pas connecté, le bouton Register apparaît sur la page d'accueil.
- Si l'utilisateur est déjà connecté, ce bouton n'est pas affiché.

Cette logique permet d'améliorer l'interface utilisateur en évitant d'afficher l'option d'inscription à un utilisateur déjà connecté.



2. Connexion des utilisateurs (Login) :

La connexion permet à un utilisateur déjà inscrit d'accéder à son compte.



○ Création du template login.html :

Un fichier login.html a été créé dans le dossier templates.

Ce formulaire demande :

- **Username**
- **Password**

```
myproject > templates > login.html > ...
1  {% load static %}
2  <!DOCTYPE html>
3  <html lang="en">
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <link rel="stylesheet" href="{% static 'admin/css/base.css' %}">
8    <link rel="stylesheet" href="{% static 'admin/css/login.css' %}">
9    <link rel="stylesheet" href="{% static 'admin/css/responsive.css' %}">
10   <title>Login</title>
11 </head>
12 <body class="login">
13   <div id="container">
14     <div id="header">
15     <h1>Login</h1>
16     </div>
17     <div id="content">
18       <div id="content-main">
19
20     <form action="login" method="post">
21       {% csrf_token %}
22
23       <div class="form-row">
24       <label for="id_username"> Username : </label>
25       <input type="text" name="username" id="id_username">
26       </div>
27
```

```
28       <div class="form-row">
29       <label for="id_password">Password : </label>
30       <input type="password" name="password" id="id_password">
31       </div>
32
33       <div class="submit-row">
34       <input type="submit" value="Log in " class = "default">
35       </div>
36     </form>
37       </div>
38     </div>
39   </div>
40 </body>
41
42 </html>
```

- ## Configuration de l'URL :

Dans urls.py, l'URL suivante est ajoutée :

```
myproject > myapp > urls.py > ...
8   path('login' , views.login , name= "login"),
9   path('logout' , views.logout , name= "logout")
```

- ## Fonction login dans views.py :

La fonction login vérifie les informations de connexion.

```
myproject > myapp > views.py > register
45  def login(request):
46      if request.method == 'POST':
47          username = request.POST['username']
48          password = request.POST['password']
49          user = auth.authenticate(username=username ,password=password )
50
51          if user is not None :
52              auth.login(request,user)
53              return redirect('/')
54          else:
55              messages.info(request,'Cred invalid')
56      else :
57          return render(request,'login.html')
58
59
60  def logout(request):
61      auth.logout(request)
62      return redirect('index')
63
```

Si les informations sont correctes :

- l'utilisateur est authentifié
- la session est ouverte
- il est redirigé vers la page d'accueil.

3. Déconnexion des utilisateurs (Logout) :

La déconnexion permet à un utilisateur de quitter son compte.

Contrairement à l'inscription et la connexion, la déconnexion ne nécessite pas de formulaire

- Fonction logout dans views.py :

```
60  def logout(request):
61      auth.logout(request)
62      return redirect('index')
63
```

Cette fonction :

- ferme la session utilisateur
- redirige l'utilisateur vers la page d'accueil.

4. Affichage dynamique dans le template :

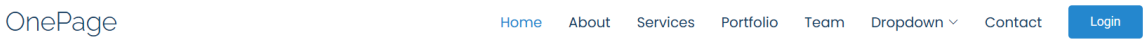
Dans index.html, l'interface change selon que l'utilisateur est connecté ou non.

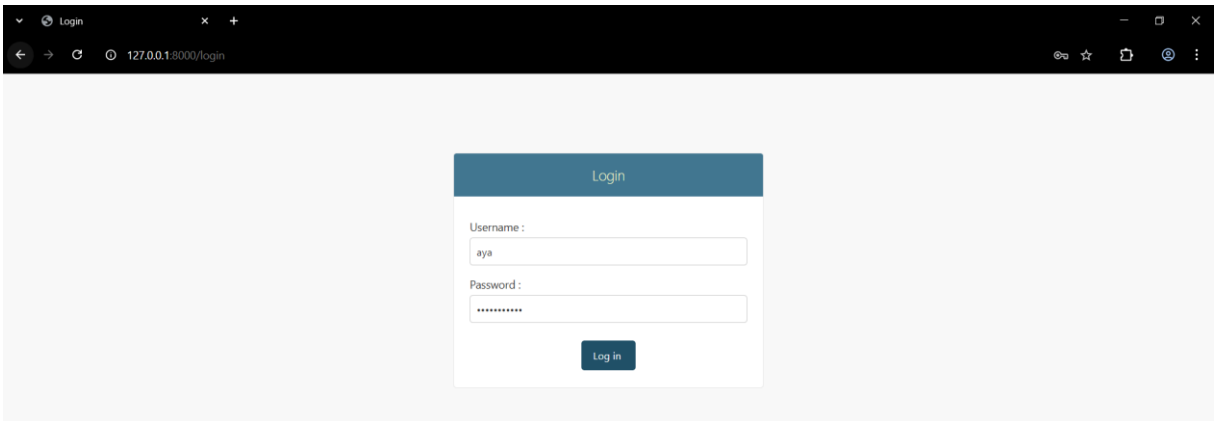
```
129        <div class="col-xl-7 col-lg-9 text-center">
130          <h3> Welcome {{user.username}}</h3>
131          <h1>One Page Bootstrap Website Template</h1>
132          <p>We are team of talented designers making websites with Bootstrap</p>
```

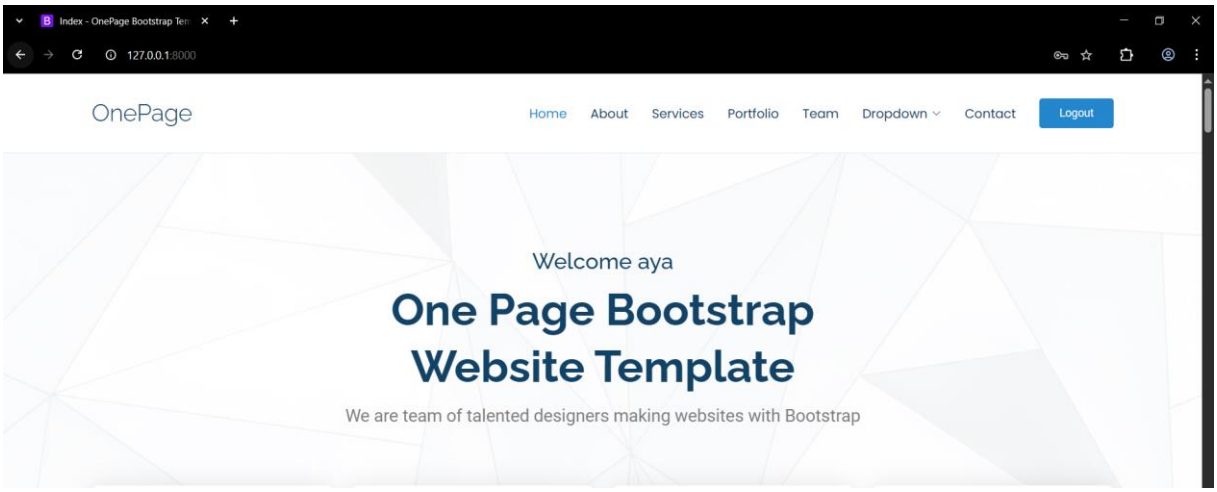
```
myproject > templates > index.html > html > body.index-page > main.main > section#hero.hero.section > div.container > div.row.gy-4.mt-5
33  <html lang="en">
70  <body class="index-page">
72    <header id="header" class="header d-flex align-items-center sticky-top">
73      <div class="container-fluid container-xl position-relative d-flex align-items-center">
109
110       {% if user.is_authenticated %}
111         <a class = "btn-getstarted" href="logout">Logout</a>
112       {% else %}
113         <a class = "btn-getstarted" href = "login">Login</a>
114       {% endif %}
115
116     </div>
117   </header>
```

Ainsi :

- si l'utilisateur est connecté → le bouton Logout apparaît
- sinon → le bouton Login est affiché.







Gestion des utilisateurs dans l'interface d'administration

Après l'implémentation du système d'authentification (inscription et connexion), les utilisateurs créés sont automatiquement enregistrés dans la base de données de Django.

Ces utilisateurs peuvent être consultés et gérés via l'interface d'administration Django accessible à l'adresse : http://127.0.0.1:8000/admin

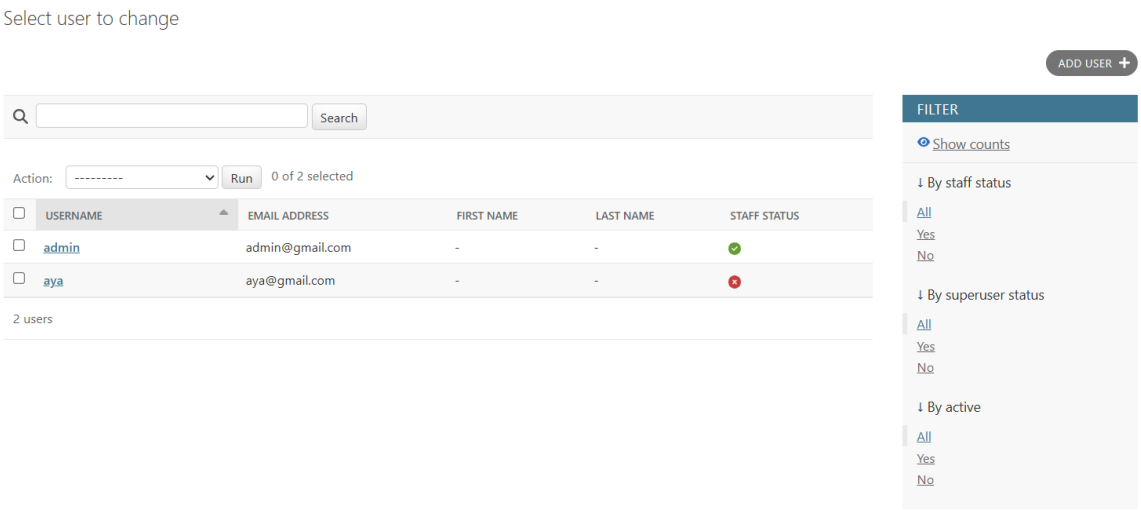
Dans la section Users, l'administrateur peut voir la liste des utilisateurs enregistrés dans l'application.

L'interface affiche plusieurs informations, notamment :

- Username : le nom d'utilisateur
- Email address : l'adresse email
- First name / Last name : les informations personnelles
- Staff status : indique si l'utilisateur est membre du staff

Dans la capture d'écran, on peut voir deux utilisateurs :

- admin : le superutilisateur créé avec la commande createsuperuser
- aya : un utilisateur créé via le formulaire d'inscription du site



Cette interface permet également :

- **d'ajouter de nouveaux utilisateurs (Add User)**
- **de modifier les informations d'un utilisateur**
- **de gérer les permissions et les rôles.**